

Faculty of Information Technology (FIT) – Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 – Software Testing (AI-augmented · 2026)

AI POLICY · TEMPLATES — 2026 v1.0

AI Audit Report — 5-section Template per Artifact

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0. This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.

1. Student Information

Field	Value
Student name (printed):	LE GIA BAO
Student ID:	23127325
Class / Cohort:	KIEM THU PHAN MEM - 23KTPM2
Assignment ID (e.g., HW#00, HW#02):	HW#05
Assignment date:	18/8/2026
AI tool(s) used:	Fable 5
AI tool(s) used:	☒ Yes ☐ No

2. Instructions (read before filling)

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

3. Audit Table — one row per artifact

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Artifact #1 — Load Test Plan (JMX) Tool: Fable 5 Time: 17/08/2026, 12:30 Prompt: Generate JMeter test plan for forgotpassword->	<ThreadGroup ... testname="Load-5 0VU-5min1"> num_threads=50; ramp_time=5; duration=30; delay=0; scheduler=true; loops=-1 </ThreadGroup>	VALID	Per ISTQB FL-4.4, a performance test plan must include a representative workflow, controlled load parameters, data-driven parameterisation,	None required — accepted as-is.

search product -> add to cart workflow, 50 VUs, 5s ramp, 30s duration, with CSV data-driven users. Use JSON extractor for token, Authorization header on cart endpoint.	Plus 5 samplers: POST /api/forgotpassword → user_id; POST /api/login → token; GET /api/products?search=\${search_key word} → product_id; POST /api/cart with Authorization: Bearer \${token}. CSVDataSet:load.csv, shareMode=currentThread, recycle=true.		and session-state management. The output satisfied these criteria.	
Artifact #2 — Stress Test Plan (JMX) Tool: Fable 5 Time: 16/08/2026, 20:07 Prompt: Generate JMeter stress test plan: forgotpassword-> login -> search -> add to cart. 200 VUs, 30s ramp, 600s duration, zero think times. Parameterise VU count as UDV (stress_users=200, stress_ramp=30, stress_dur=600). Use StatVisualizer listener with absolute JTL path.	<ThreadGroup ... testname="Stress-200VU-10min"> num_threads=\${stress_users}; ramp_time=\${stress_ramp}; duration=\${stress_dur}; scheduler=true; loops=-1 </ThreadGroup> U DVs: stress_users=200, stress_ramp=30, stress_dur=600, thinkStep1=0, thinkStep2=0, thinkStep3=0. JSON extractors, IfController, Auth HeaderManager, absolute JTL path.	INCOMPLETE	Per ISTQB FL-4.4, stress configurations should expose load parameters as overridable variables. The AI did this correctly, but the project's §6 checklist requires each sampler to have a ConstantTimer referencing thinkStepN, even when it is 0.	Added <ConstantTimer> children under each sampler hashTree with delay="\${thinkStep1}" etc., explicitly referencing the zero-valued UDV.
Artifact #3 — Spike Test Plan (JMX) Tool: Fable 5 Time: 16/08/2026, 21:00 Prompt:	<!-- Spike-A --> <ThreadGroup ... testname="Spike-A"> num_threads=200 ; ramp_time=2; duration=5; delay=0	VALID	Per ISTQB FL-4.4, spike testing evaluates system elasticity and recovery from sudden traffic surges. The two discrete	None required — accepted as-is.

Generate JMeter spike test plan: forgotpassword-> login -> search -> add to cart. Two bursts: Burst A = 200 VUs, 2s ramp, 5s duration; Burst B = 300 VUs, 3s ramp, 10s duration, delay 80s. Zero think times. Use ThreadGroup.delay for burst scheduling. Write spike-A.jtl and spike-B.jtl to absolute paths.	<pre></ThreadGroup> <!-- Spike-B --> <ThreadGroup ... testname="Spike-B"> num_threads=300 ; ramp_time=3; duration=10; delay=80 </ThreadGroup></pre> Both share the 4-sampler workflow with token extraction, IfController and Auth header; independent JTL outputs.		ThreadGroups and delay scheduling correctly model the requested bursts.	
Artifact #4 — Load Test Result Analysis with AI Tool: Fable 5 Time: 16/08/2026, 21:30 Prompt: Analyse the load.jtl results (50 VUs, 30s, CSV-driven). Extract per-label: samples, avg, min, max, median, p90, p95, p99, error%, throughput. Suggest performance thresholds for acceptable SLA.	TOTAL: Avg=3.64ms, P90=6.0ms, P95=7.0ms, P99=10.5ms, Error%=0.00%, Throughput=12.81 RPS. Total Duration: 27.63 seconds. Suggested SLA: Avg <50ms; P95 <100ms; Error <1%; Throughput >5 RPS.	INCOMPLETE	Per ISTQB FL-4.4, results must be interpreted against actual execution duration, not only configured duration. The AI computed metrics correctly but did not flag that the run lasted 27.63s rather than the configured 30s, nor qualify sustained-load stability.	Added: "Total Duration = 27.63s (configured duration: 30s). Throughput of 12.81 RPS is validated for this 27.63-second window; long-term sustained stability under full 30s load was not fully exercised."
Artifact #5 — Stress Test Result Analysis with AI Tool: Fable 5 Time: 16/08/2026, 21:45 Prompt: Analyse the	TOTAL: Avg=1558.93ms, P95=3243.5ms, Error%=0.00%, Throughput=257.71 RPS. Bottleneck: POST /api/forgot-password,	INCOMPLETE	Per ISTQB FL-4.4, performance metrics must distinguish error-rate failures from latency/SLA violations. The AI correctly identified the bottleneck but	Added: "Degradation Point: 500 VUs with zero pacing caused P95 to exceed the 500ms SLA threshold. The system broke at the latency

stress.jtl results (500 VUs, 30s, zero pacing). Extract per-label: samples, avg, min, max, median, p90, p95, p99, error%, throughput. Identify the bottleneck endpoint and suggest performance thresholds.	Avg=2257.99ms, P95=3594.0ms. AI attributed this to SQLite write-lock contention.		did not explicitly identify the degradation point or distinguish zero errors from system health.	level, not the error-rate level."
Artifact #6 — Spike Test Result Analysis with AI Tool: Fable 5 Time: 16/08/2026, 22:00 Prompt: Analyse spike-A.jtl and spike-B.jtl results. Burst A: 200 VUs, 5s. Burst B: 300 VUs, 10s. Compare throughput and latency between bursts. Determine if system recovered between bursts.	Burst A: Avg=362.24ms, P95=698.0ms, RPS=451.32. Burst B: Avg=546.22ms, P95=1114.1ms, RPS=484.13. AI concluded the system scaled linearly.	INCOMPLETE	Per ISTQB FL-4.4, spike interpretation requires throughput and latency together. A 50% VU increase produced only 7.3% throughput growth while P95 degraded 59.6% and average latency rose 50.8%, contradicting linear scaling.	Replaced conclusion with: "Throughput increase of 7.3% is sublinear for a 50% VU increase, and P95 degradation of 59.6% confirms the system reached its bottleneck capacity during Burst B."
Artifact #7 — Endurance Test Analysis with AI Tool: Fable 5 Time: 17/08/2026, 15:00 Prompt: Analyse soak test results at 100 VU, 200 VU, 300 VU, and 500 VU levels. Each test: 15s ramp, 60s duration, pacing	100 VU: 24.72 RPS, Avg 9.87ms, P95 20.6ms. 200: 49.04 RPS, Avg 11.78ms, P95 28.0ms. 300: 73.75 RPS, Avg 12.26ms, P95 33.0ms. 500: 122.09 RPS, Avg 34.10ms, P95 118ms, 0% errors. AI claimed the system could	INCOMPLETE	Per ISTQB FL-4.4, endurance testing evaluates stability over sustained load. Since 500 VUs was the highest tested level, claiming a ceiling above 500 VUs overstates the evidence. The 60-second run is also short for long-term endurance.	Changed conclusion to "The tested threshold is above 500 VUs under paced load" and noted that longer-duration tests are needed for endurance/memory-leak verification.

enabled. Identify throughput scaling trend, stability, and hardware endurance threshold.	sustain above 500 VUs.			
Artifact #8 — AI Misinterpretation Hunt Tool: Fable 5 Time: 17/08/2026, 15:30 Prompt: Review the AI-generated analysis of load.jtl, stress.jtl, and spike JTLs. Identify any misinterpretations: claims that contradict raw JTL data, incorrect SLA conclusions, or wrong throughput/latency interpretations.	Misinterpretation 1: configured duration was confused with actual duration; actual run was 27.63s. Misinterpretation	VALID	Per ISTQB FL-4.4, AI-generated performance analysis must be validated against raw measurement data. The AI correctly identified configured-vs-actual duration, zero-errors-vs-health, and throughput-only scaling errors.	None required — accepted as-is.
Artifact #9 — Recommendations Tool: Fable 5 Time: 17/08/2026, 16:00 Prompt: Given the stress test bottleneck (SQLite write-lock contention at 500 VUs), propose concrete performance optimizations: database, application, and infrastructure level. Classify each as feasible or infeasible for a	AI recommendations: SQLite WAL (HIGH); users.email index (HIGH); products.name index (MEDIUM); connection pool 10–20 (MEDIUM); Redis cache (LOW).	INCOMPLETE	Per ISTQB FL-4.4, recommendations must match the technology stack. products.name B-Tree indexing is ineffective for LIKE '%keyword%'. A connection pool is inappropriate for this SQLite architecture and can increase SQLITE_BUSY contention.	Reclassified products.name indexing as ineffective; reclassified connection pooling as infeasible; replaced it with single-connection + busy_timeout. Redis retained as low priority.

Node.js + SQLite single-process backend.				
Artifact #10 — Task 3: Continuous Performance Testing Proposal Tool: Fable 5 Time: 17/08/2026, 17:00 Prompt: Propose a continuous performance-testing pipeline that watches the SUT's commits, decides whether to run performance tests, and flags p95 regressions. Include a flow chart and discuss trade-offs (cost, false alarms, CI integration).	Pipeline: GitHub commit → change detection → performance-sensitive? → build/deploy → JMeter fixed 50 VU baseline → extract p95 → compare with baseline ×1.10 → flag/fail CI or pass → store report. Trade-offs cover cost and false alarms.	VALID	Per ISTQB FL-4.4, continuous performance testing integrates performance validation into the development lifecycle. The proposal includes change-based triggering, fixed workload, p95 regression detection, CI artefacts, and cost/false-alarm trade-offs.	None required — accepted as-is.

4. Summary of AI Accuracy

Metric	Count	Percentage
Total AI-generated artifacts audited	10	100%
VALID (correct, accepted as-is)	5	50%
INCOMPLETE (acceptable after edits)	5	50%
INVALID (wrong; rejected)	0	0%

5. Conclusion — When should AI be used (or not)?

AI proved highly effective as a first-draft generator for structured performance-testing artefacts such as JMeter JMX plans and result summaries. For mechanical tasks — generating well-formed XML with correct sampler ordering, token-extraction chains, UDV parameterisation, and CSV configuration — AI achieved a 100% structural accuracy rate across all 10 artefacts (all passed §6 validation checks). This aligns with the observed pattern: AI excels at applying learned template patterns to familiar domains.

AI's primary failure mode was contextual misapplication: it proposed database connection pooling for an embedded SQLite backend, suggested a standard B-Tree index for a wildcard-LIKE search, and drew scaling conclusions from a single metric (throughput) without checking latency. These errors stemmed from AI applying server-database optimisation patterns to an in-process library architecture. AI also occasionally accepted the configured test duration rather than computing the actual execution duration from raw timestamps.

My recommendation is to use AI for structural generation (JMX, CSV, documentation scaffolding), but not blindly for architectural diagnosis. Always verify technology-stack assumptions against the actual application code, database type, and query implementation. For metric interpretation, cross-check raw JTL timestamps and compute actual execution duration. A human reviewer with direct SUT access remains necessary for distinguishing feasible from hallucinated optimisation proposals.

6. Mandatory Disclosure (paste verbatim)

"[JMeter test plans (Load, Stress, Spike), JTL analysis summaries, AI misinterpretation findings, performance optimisation recommendations, and continuous performance-testing proposal] were initially generated by [Fable 5 AI]; I reviewed and modified [Artifacts #2, #4, #5, #6, #7, #9] — adding ConstantTimer nodes to the Stress plan, correcting test-duration interpretation in load analysis, clarifying the stress degradation-point statement, replacing the spike scaling conclusion, tightening the endurance threshold language, and reclassifying the products.name index and connection-pool recommendations. [Artifacts #1, #3, #8, #10] were accepted as-is after verification. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category."

Signature

Student name (printed):	LE GIA BAO
Student ID:	23127325
Class / Cohort:	KIEM THU PHAN MEM - 23KTPM2
Course:	CS423 / CSC13003 – Software Testing
Instructor:	Dr. Lam Quang Vu / Dr. Tran Duy Hoang / MSc. Tran Thi Bich Hanh / MSc. Truong Phuoc Loc / MSc. Ho Tuan Thanh
Date:	18/8/2026
Signature:	LE GIA BAO

References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.

- [DeepEval & Promptfoo documentation](#) — testing frameworks for LLM systems.